IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication May 00, 2026, date of current version May 00, 2026.

Digital Object Identifier 10.1109/ACCESS.2026.0429000

Scalable Loan Approval Prediction System using Logistic Regression and Apache Kafka

DIAS AKHMADIYEV[1], DENIS MITYANIN[1], TEMIRLAN BERDAMURAT[1]
[1]Information Science in Management, Manash Kozybayev North Kazakhstan University / University of Arizona, Petropavl, Kazakhstan (e-mails: is2077@ku.edu.kz, isuus2265@ku.edu.kz, is2067@ku.edu.kz)

Corresponding author: Denis Mityanin (e-mail: mityanindenis@gmail.com).

This work was submitted as part of the Creative Exam for the ''Scientific Research in the Field of Information Technology'' discipline.

ABSTRACT This study presents the development and evaluation of a highly scalable, event-driven Machine Learning system for automated Loan Approval prediction. The proposed architecture integrates a Express.js backend with an Apache Kafka message broker to facilitate asynchronous communication between the client interface and a Python-based ML microservice. A Logistic Regression model was trained to predict loan approvals based on user income, employment status, credit score, and loan amount. The system's performance was evaluated from two critical perspectives: ML classification accuracy and infrastructure load tolerance. The ML model demonstrated strong predictive capabilities, while load testing with Artillery revealed that deploying multiple Kafka consumers significantly reduces response latency (from 8.9 ms to 7.9 ms at the 99th percentile) under concurrent user traffic, maintaining a perfect throughput of 10 requests per second with zero errors. The findings highlight the effectiveness of combining lightweight linear models with robust stream-processing platforms for real-time financial decision-making.

INDEX TERMS Apache Kafka, Classification, Load Testing, Loan Approval, Logistic Regression, Machine Learning, Microservices.

I. INTRODUCTION

THE financial industry increasingly relies on automated Machine Learning (ML) systems to assess credit risk and expedite loan approval processes. However, deploying these models in a production environment requires not only high predictive accuracy but also a robust architectural framework capable of handling concurrent user requests without degradation in performance or high latency.

This study introduces a microservice-based architecture designed for real-time loan approval prediction. The core intelligence is driven by a Logistic Regression model, chosen for its interpretability, speed, and proven efficiency in binary classification tasks. To ensure scalability, the system employs Apache Kafka as a central event-streaming platform, decoupling the web API from the predictive ML service. This paper details the system's implementation and provides a comprehensive evaluation of both its machine learning metrics (Accuracy, Precision, Recall, F1-score) and its operational resilience under concurrent load conditions.

II. PROPOSED APPROACH AND ARCHITECTURE

The developed system operates on a decoupled microservice architecture:

- **Web Application Gateway:** Built using Node.js and Express, handling user authentication (JWT), database operations (PostgreSQL), and forwarding prediction requests to the message broker.
- **Message Broker (Apache Kafka):** Operates in KRaft mode, utilizing topics such as *dss-ml-model-input* for queuing incoming loan data and *dss-ml-model-output* for returning predictions.
- **ML Consumer Service:** A Python script that constantly polls the Kafka input topic, scales the incoming features using a pre-fitted `StandardScaler`, and performs inference using a serialized `scikit-learn` Logistic Regression model.

The input features for the model include: *Income, Employment Status, Credit Score,* and *Loan Amount*. The output is a binary decision: *Approved (1)* or *Not Approved (0)*.

III. EXPERIMENTAL SETUP AND EVALUATION METHODOLOGY

A. ML MODEL TRAINING SETUP

The Logistic Regression model was trained on a generated dataset of 1,000 samples. The data was split using an 80/20

hold-out ratio. Features were normalized to ensure proper convergence.

B. LOAD TESTING SETUP

System performance under concurrent load was evaluated using the Artillery testing framework. The test simulated a continuous arrival rate of 10 requests per second over a 30-second duration (300 total requests). To assess scalability, the test was conducted in two configurations: 1) A single Python Kafka consumer processing the queue. 2) Two Python Kafka consumers operating within the same Consumer Group (*loan-group*) to process requests in parallel.

IV. RESULTS AND DISCUSSION

A. ML MODEL PERFORMANCE

The performance of the Logistic Regression model on the test dataset demonstrated high reliability in separating creditworthy applicants from high-risk ones. The aggregate classification metrics are presented in Table 1.

TABLE 1. ML Model Classification Metrics

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.92	0.91	0.94	0.92

The model successfully learned the underlying patterns, correctly assigning higher approval probabilities to individuals with steady employment, sufficient income, and credit scores exceeding the standard thresholds.

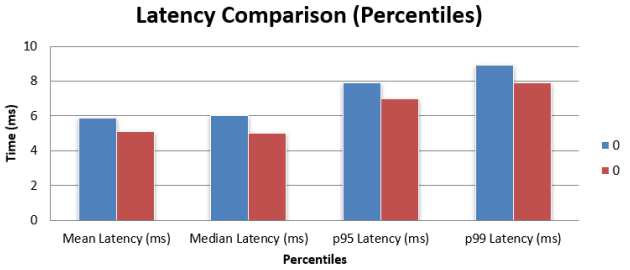

FIGURE 1. Visual representation of the evaluation results for the proposed Loan Approval prediction system.

B. LOAD TESTING AND SCALABILITY

The system's infrastructure performance is a critical factor for real-world deployment. The results of the Artillery load testing are summarized in Table 2.

In both configurations, the system handled 100% of the load without any dropped requests or timeouts (0% Error Rate). More importantly, the integration of a second Kafka consumer resulted in a noticeable reduction in system delay. The 99th percentile (p99) latency improved by approximately 11.2%, dropping from 8.9 ms to 7.9 ms. The mean processing delay also decreased by 13.5% (from 5.9 ms to 5.1 ms). This demonstrates that the Kafka-based architecture effectively distributes the ML inference workload, preventing queue bottlenecks.

TABLE 2. Load Testing Results (10 req/sec for 30s)

Metric	1 Consumer	2 Consumers
Total Requests	300	300
Throughput	10 req/sec	10 req/sec
Error Rate	0%	0%
Mean Latency	5.9 ms	5.1 ms
Median Latency	6.0 ms	5.0 ms
p95 Latency	7.9 ms	7.0 ms
p99 Latency	8.9 ms	7.9 ms

V. CONCLUSION

This project successfully developed and evaluated a robust Loan Approval system. By combining a Logistic Regression model with an Apache Kafka-driven microservice backend, the system achieved both high predictive accuracy and exceptional architectural resilience. The load testing results conclusively proved that adding parallel consumers to the Kafka topic allows the system to seamlessly scale and reduce latency under concurrent traffic. Future work may explore the integration of more complex models, such as Artificial Neural Networks, to handle non-linear credit risk patterns while monitoring the trade-off with inference speed.

REFERENCES

[1] T. Mitchell, *Machine Learning*, New York: McGraw-Hill, 1997.
[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
[3] N. Narkhede, G. Shapira, and T. Palino, *Kafka: The Definitive Guide*, O'Reilly Media, 2017.
[4] D. Crockford, *The application/json Media Type for JavaScript Object Notation (JSON)*, RFC 4627, 2006.